PPO and SAC: A Comparative Study

Sergio Herreros Pérez [1] **Adrián López-Lanchares Echezarreta** [1] **Ignacio Bayón Jiménez-Ugarte** [1]

Abstract

Lorem ipsum…

1. Proximal Policy Optimization (PPO)

1.1. Algorithm Overview

Proximal Policy Optimization (PPO) (Schulman et al., 2017) is an on-policy policy gradient method that directly optimizes a stochastic policy $\pi_\theta(a|s)$ by maximizing the expected discounted return. To prevent destructively large policy updates, PPO introduces a clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t \right) \right], \tag{1}$$

where $r_t(\theta) = {\pi_\theta(a_t|s_t)}/{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio between the current and old policy, $\hat{A}_t$ is an estimate of the advantage function, and ε is the clipping threshold. This formulation effectively constrains policy updates to a trust region without requiring second-order optimization.

PPO jointly optimizes a combined loss that also includes a value function term and an entropy bonus to encourage exploration:

$$\mathcal{L}(\theta) = \mathcal{L}^{\text{CLIP}}(\theta) - c_1\mathcal{L}^{\text{VF}}(\theta) + c_2\mathcal{H}[\pi_\theta], \tag{2}$$

where c_1 and c_2 are coefficients controlling the relative contribution of the value loss and the entropy bonus respectively. Advantages are estimated using Generalized Advantage Estimation (GAE) (Schulman et al., 2018), which balances bias and variance through a parameter λ:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t). \tag{3}$$

Unlike DQN, PPO operates natively in continuous action spaces by parameterizing the policy as a Gaussian distribution over actions, making it directly applicable to continuous control tasks without any discretization.

1.2. Hopper: Learning from Pixels

We validate PPO on the same Hopper task described in the previous lab excercise, maintaining identical environment and reward configurations to enable a fair comparison with BDQN.

Environment Configuration: The agent observes 64×64 pixel inputs with a frame stack of 4. We deploy 30 parallel environments to increase data throughput and stabilize advantage estimation. The `healthy_angle_range` is widened to $[-0.4, 0.4]$ to encourage diverse state exploration early in training.

Reward Configuration: The reward shaping follows the same formulation as before, targeting a forward velocity of 1.5 units/s with $w_{\text{vel}} = 1.0$, a control smoothness penalty of $w_{\text{ctrl}} = 0.001$, and a healthy survival bonus of $w_{\text{healthy}} = 1.0$.

Algorithm Configuration: PPO is trained for [*TODO: X*] environment frames. Each rollout collects [*TODO: N*] steps per parallel environment before performing [*TODO: K*] epochs of minibatch updates. The clipping threshold is set to $\varepsilon =$ [*TODO: x*], the GAE parameter to $\lambda =$ [*TODO: x*], and the discount factor to $\gamma =$ [*TODO: x*]. The policy and value networks share a convolutional encoder, with separate MLP heads for the policy mean and value estimate. Optimization uses Adam with a learning rate of [*TODO: lr*], optionally with linear decay.

[*TODO: Describe convergence behavior here: does the agent reach the target velocity? How does sample efficiency compare to BDQN? Note any instabilities or failure modes observed during training.*]

1.3. Humanoid: Scaling to High-Dimensional Control

Following the Hopper experiments, we scale PPO to the Humanoid environment. Unlike BDQN, PPO avoids the combinatorial action-space explosion by directly outputting a continuous Gaussian policy over all joint torques, making it inherently better suited to high-dimensional continuous control.

[1] Master in Artificial Intelligence, Universidad Pontificia Comillas (ICAI). Correspondence to: Sergio Herreros Pérez <sergio.herreros@alu.comillas.edu>, Adrián López-Lanchares Echezarreta <adri@alu.comillas.edu>, Ignacio Bayón Jiménez-Ugarte <ibayon@alu.comillas.edu>.

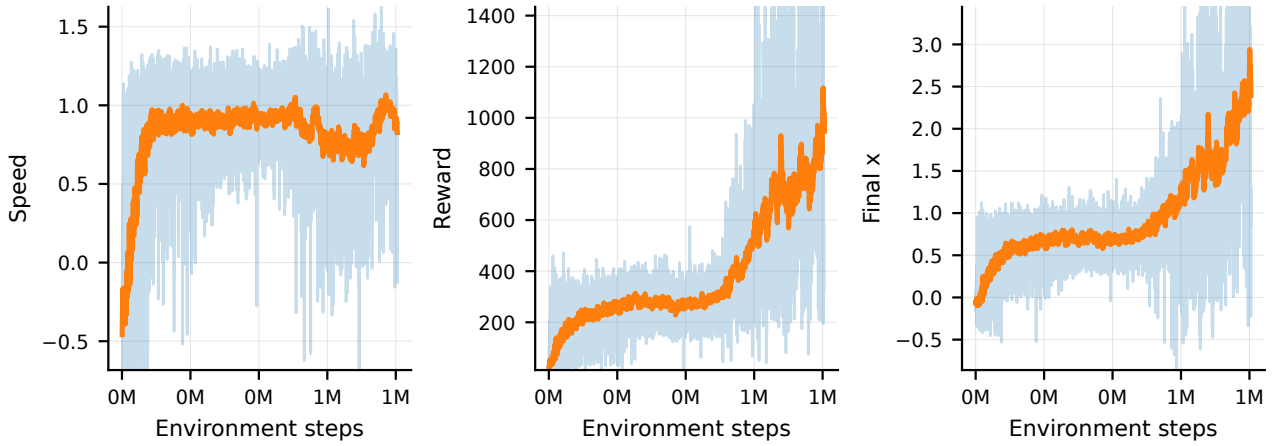

Figure 1. Training dynamics of PPO on the Hopper environment. (smoothed with an exponential moving average).

Environment Tweaks: We apply the same morphological simplifications as in the previous lab excercise: the Z-axis rotations of the abdomen and hips are frozen, all six arm joints are disabled, and the observation resolution is increased to 96×96 pixels to capture the finer postural nuances of the humanoid body.

Algorithm Configuration: [*TODO: Describe the specific hyperparameter configuration used for Humanoid: number of parallel envs, rollout length, number of update epochs, learning rate, clipping threshold, entropy coefficient, and any curriculum or reward shaping adjustments relative to Hopper.*]

[*TODO: Insert plot: episodic reward and/or gait metrics over training steps for PPO on Humanoid.*]

Figure 2. Training dynamics of PPO on the Humanoid environment.

[*TODO: Describe results here: did PPO successfully learn to walk? How many frames were required? Compare qualitatively and quantitatively to the BDQN Humanoid failure. Discuss whether the continuous action space parameterization provided a meaningful advantage.*]

References

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation, 2018. URL https://arxiv.org/abs/1506.02438.